Week 7 · Saving and history

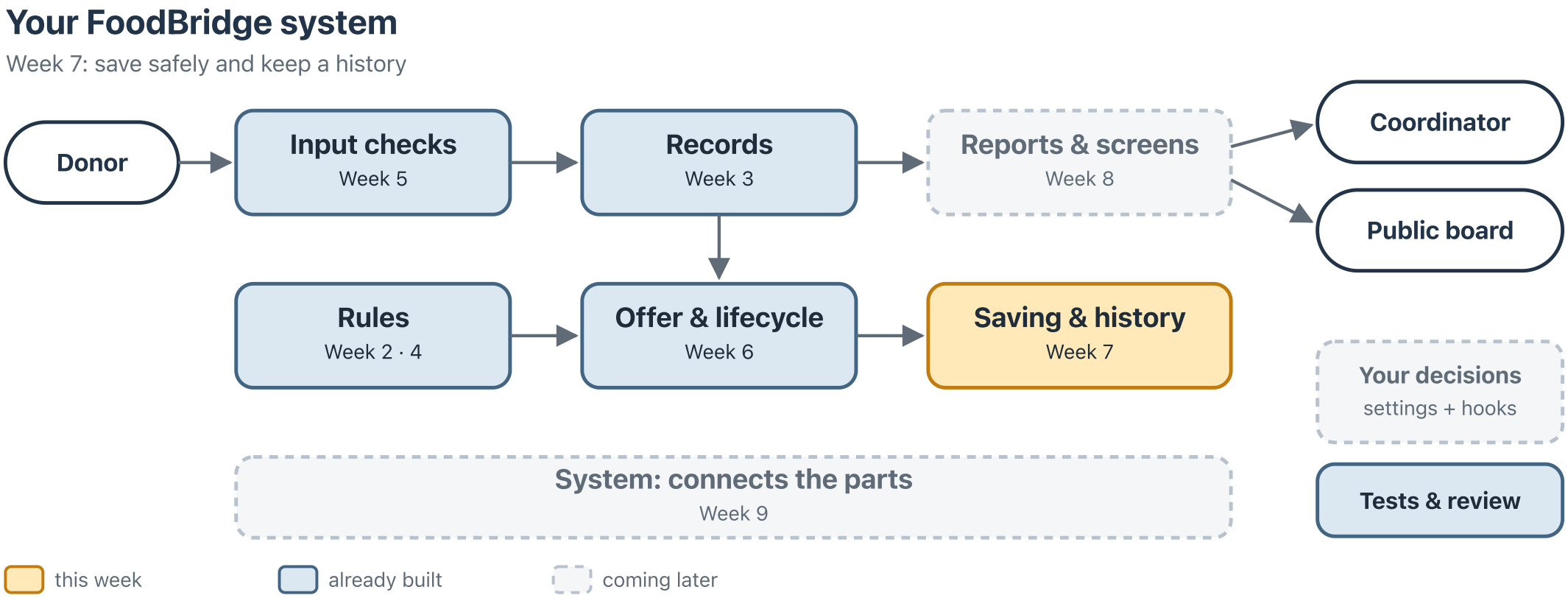
Programming for Information Systems

Save safely, keep a history, explain what happened

This week

Question	How does a system remember, and how can it explain what happened?
Python	`open` with `with`, text files, `json`, JSON Lines, `os.path.exists`
Principle 7	Never lose data
You build	`to_record` and `from_record`

Your system, by Week 10



Memory forgets, records must not

- A program's memory is a **whiteboard**: wiped when the program stops.
- **Persistence:** records are saved in a file, so they are there tomorrow.
- A shop's **sales book**: every sale is a new line in ink, with the time.

> A mistake is fixed with a **new line**, not by tearing out the page.

Audit history: who changed what, and when

```
#10 moved F002 new -> approved for Riverside Community Kitchen
#12 moved F003 new -> rejected (no cold storage for fruit today)
#15 moved F002 picked_up -> delivered
```

- **Append-only:** we only add events; we never edit or delete them.
- A donor asks: "*Why were my bananas rejected?*" The history answers.

Python foundations

Files, JSON and history

Write and read a text file

```
with open("shopping.txt", "w", encoding="utf-8") as file:
    file.write("rice\n")
    file.write("eggs\n")

with open("shopping.txt", encoding="utf-8") as file:
    for line in file:
        print("-", line.strip())
```

- `with` closes the file for you, even when something goes wrong.
- `"\n"` ends a line; `strip()` removes it again.

🤔 Predict: `"w"` or `"a"`?

```
with open("shopping.txt", "w", encoding="utf-8") as file:
    file.write("bananas\n")
with open("shopping.txt", "a", encoding="utf-8") as file:
    file.write("bread\n")
```

- The file had *rice* and *eggs*. What is in it now?
- `"w"` **empties** the file the moment it opens. No warning.
- `"a"` **adds** to the end.

Does the file exist?

```python
import os

print(os.path.exists("shopping.txt"))   # True
print(os.path.exists("homework.txt"))   # False
```

- Reading a missing file stops the program (`FileNotFoundError`).
- First day of a new system: no file yet → start with `[]`.

JSON: data as text, and back

```
import json

phone = {"owner": "Dara", "credit_usd": 5, "active": True}
text = json.dumps(phone)          # data -> text
back = json.loads(text)           # text -> data
print(back["credit_usd"] + 1)     # 6: still a number
```

- JSON keeps numbers as numbers (CSV gave us only text).
- `dump` / `load` (no *s*) do the same with a file.

JSON Lines: one event per line

```
def log_event(path, event):
    with open(path, "a", encoding="utf-8") as file:
        file.write(json.dumps(event) + "\n")

log_event("history.jsonl", {"time": "09:00", "event": "top up"})
log_event("history.jsonl", {"time": "09:30", "event": "bought data"})
```

- Mode `"a"`: old lines are never rewritten.
- Run it twice and the history grows. That is correct for a history.

Read and judge: the safe save

Saving safely: never leave a half-written file



If the program stops during step 1, the old offers.json is still complete.

Risky: open('offers.json', 'w') empties the file first, then writes.

Risky vs safe

Risky (short, common in AI code)

```
def risky_save(records, path):
    with open(path, "w") as file:
        json.dump(records, file)
```

The old file is emptied **first**.

Safe (the kit's `save_records`)

```
temp_path = path + ".tmp"
with open(temp_path, "w") as file:
    json.dump(records, file)
os.replace(temp_path, path)
```

The old file is replaced only when the new one is complete.

⚖️ Good or risky? An AI's `save_day`

Version A (from an AI assistant)

```
data = []
total = 0
for offer in offers:
    data.append({"id": offer.id,
                 "food": offer.food,
                 "portions": offer.portions})
    total += offer.portions
data.append({"total_portions": total})
with open("offers.json", "w") as f:
    json.dump(data, f)
with open("history.jsonl", "w") as f:
    for event in history[-10:]:   # "keep it small"
        f.write(json.dumps(event) + "\n")
```

Version B

```
records = []
for offer in offers:
    records.append(to_record(offer))
bridge.storage.save_records(records, "offers.json")
for event in new_events:
    bridge.storage.append_event("history.jsonl", event)
```

Find **four** problems in A. Who is harmed?

Principle 7 · Never lose data

- **Save safely:** write the new file completely, then swap it in.
- **Keep a history:** add events; never edit or delete them.
- **Be able to explain** what happened, to whom, and when.

> When you review AI code, ask: *what happens on a bad day?*

You build: `to_record` and `from_record`

```
def to_record(offer):        # Offer -> dictionary, before saving
    record = {}
    record["id"] = offer.id
    ...                      # every field in bridge.models.FIELDS
    return record

def from_record(record):     # dictionary -> Offer, after loading
    offer = bridge.Offer(record["id"], record["donor"], ...)
    offer.status = record["status"]     # a new Offer starts as "new"
    offer.partner = record["partner"]
    return offer
```

Plug in: the system remembers

```
records = []
for offer in my_system.offers:
    records.append(to_record(offer))
bridge.storage.save_records(records, "my_offers.json")

first = bridge.System(MY_SETTINGS, data_dir="my_data")
first.run_day(show=False)       # saved after every change
second = bridge.System(MY_SETTINGS, data_dir="my_data")
print(second.report())          # everything is back
```

🧭 Your decision · what goes into the history?

```
LOG_EVENTS = ["added", "input_rejected", "moved"]
```

- Remove `"input_rejected"` → the summary says **"not logged"**, not 0.
- **Accountability:** more events, more you can explain.
- **Noise:** more to read; the one event that matters is easier to miss.

This week's tasks

- ⭐ **Core:** P1–P4, Good or risky, `to_record` and `from_record` with all ✅, the decision log.
- ⭐⭐ **Stretch:** count events by type; the story of one offer from the history.
- 🔎 **Research 6 (5%):** check an AI answer against an original source. Share it in the Week 8 warm-up.

Next week · Reports and screens

The same records, shown to **different people**. Who sees what? How do we keep numbers honest and private data private?